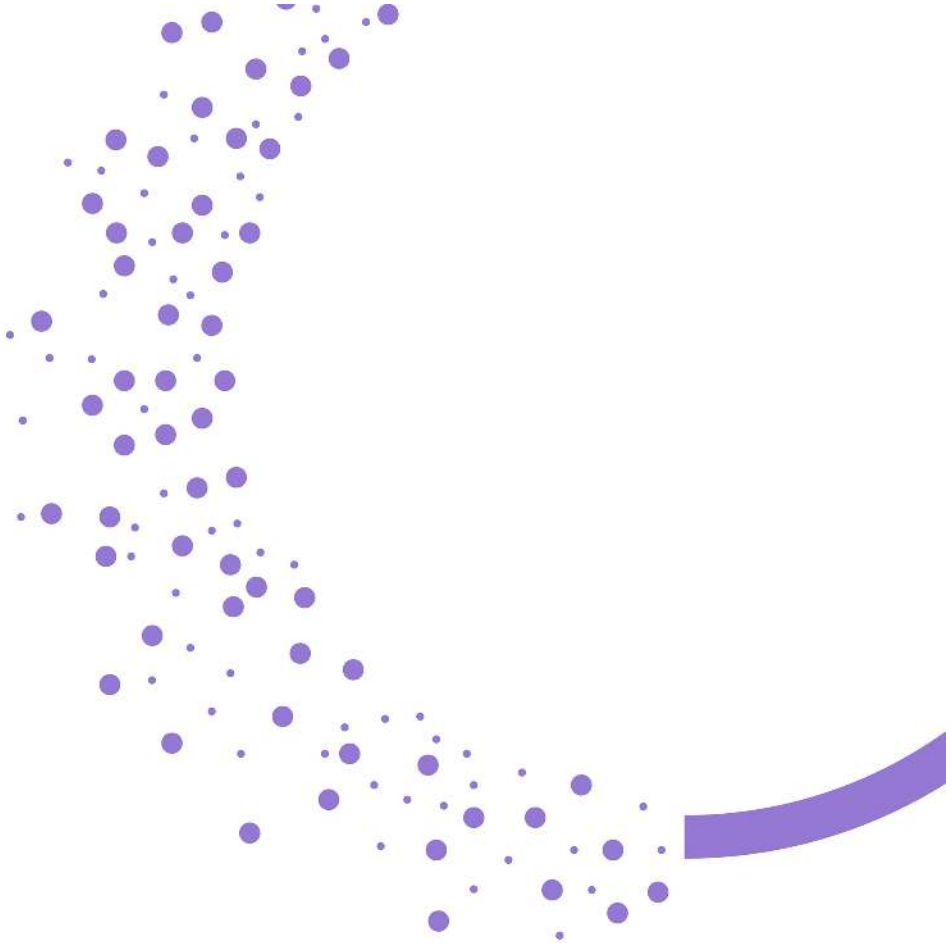
Counter Galois Onion (CGO)

A New Onion Encryption Scheme for Tor

Jean Paul Degabriele, Alessando Melloni, Jean-Pierre Münch, Martijn Stam

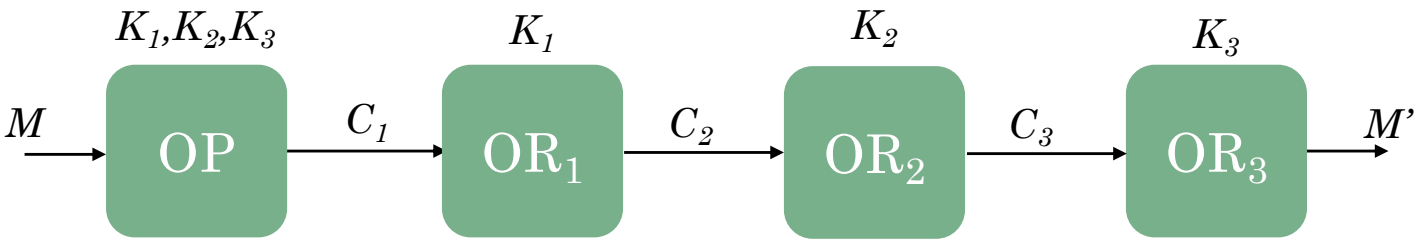
Technology Innovation Institute

Tor's Onion Encryption and its Limitations



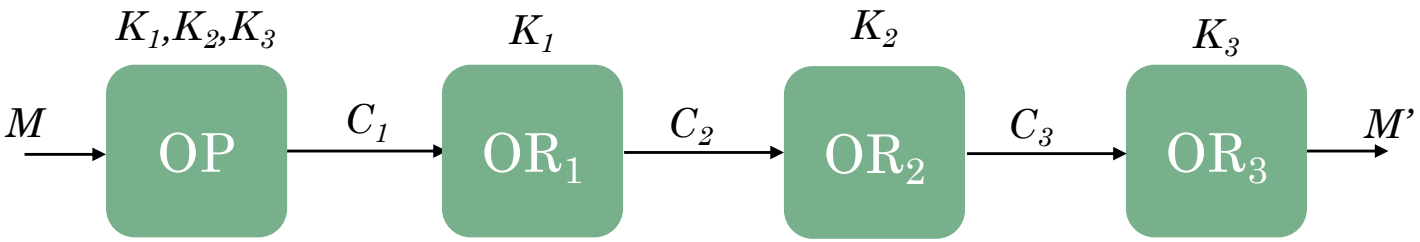
tii.ae

Onion Encryption in Tor



- In Tor, data flows through **circuits** consisting of an **Onion Proxy (OP)** and a sequence of **Onion Routers (OR)**.

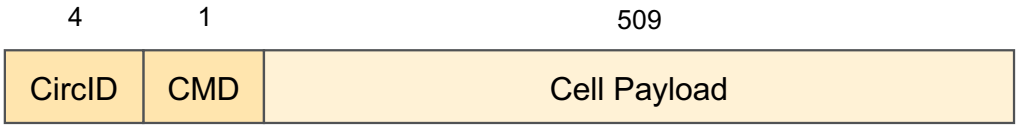
Onion Encryption in Tor



- In Tor, data flows through **circuits** consisting of an **Onion Proxy (OP)** and a sequence of **Onion Routers (OR)**.
- Circuits are **analogues of symmetric channels**, where the Onion Proxy shares a symmetric key with each Onion Router.
- Each Onion Router **"peels off"** a layer of encryption, **decorrelating incoming ciphertexts** from **outgoing ciphertexts**.

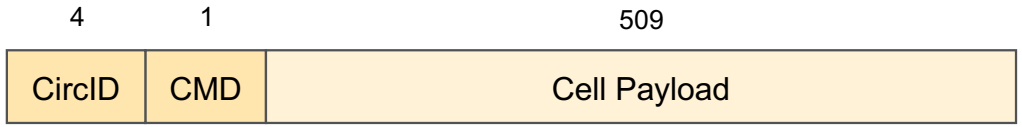
Relay Cell Format and Processing [DMS04]

- Cells are 514 bytes (v4+)



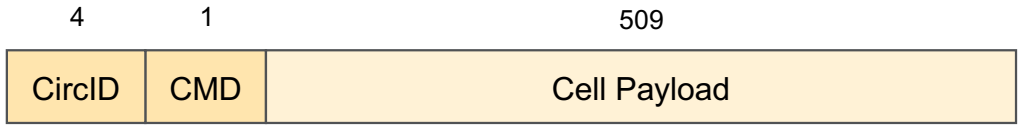
Relay Cell Format and Processing [DMS04]

- Cells are 514 bytes (v4+)
- **CircID**: Circuit Identifier



Relay Cell Format and Processing [DMS04]

- Cells are 514 bytes (v4+)
- **CircID**: Circuit Identifier
- **CMD**: Cell type - RELAY (3) or RELAY_EARLY (9)



Relay Cell Format and Processing [DMS04]

- Cells are 514 bytes (v4+)
- **CircID**: Circuit Identifier
- **CMD**: Cell type - RELAY (3) or RELAY_EARLY (9)

4	1	509
CircID	CMD	Cell Payload

4	1	1	2	2	4	2	498
CircID	CMD	rCMD	Rx	SID	Digest	Len	Data

Relay Cell Format and Processing [DMS04]

- Cells are 514 bytes (v4+)
- **CircID**: Circuit Identifier
- **CMD**: Cell type - RELAY (3) or RELAY_EARLY (9)
- **Rx**: Recognised field (0x0000)

4	1	509
CircID	CMD	Cell Payload

4	1	1	2	2	4	2	498
CircID	CMD	rCMD	Rx	SID	Digest	Len	Data

Relay Cell Format and Processing [DMS04]

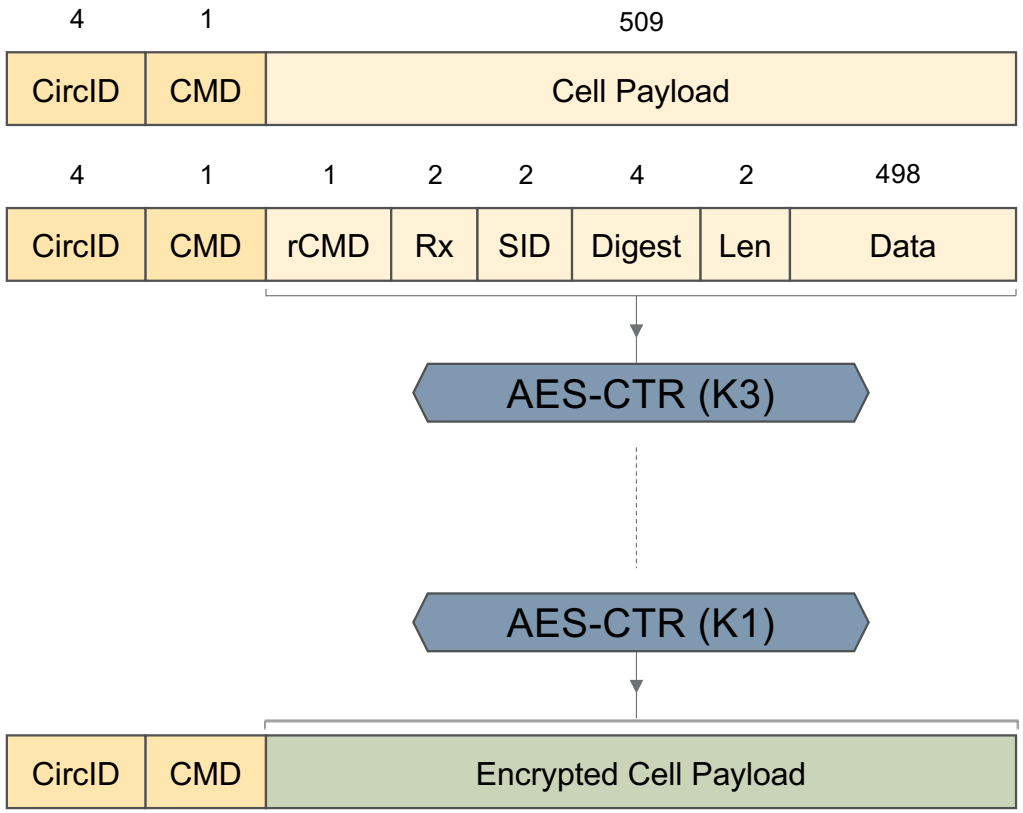
- Cells are 514 bytes (v4+)
- **CircID**: Circuit Identifier
- **CMD**: Cell type - RELAY (3) or RELAY_EARLY (9)
- **Rx**: Recognised field (0x0000)
- **Digest**: seeded running hash (truncated SHA-1)

4	1	509
CircID	CMD	Cell Payload

4	1	1	2	2	4	2	498
CircID	CMD	rCMD	Rx	SID	Digest	Len	Data

Relay Cell Format and Processing [DMS04]

- Cells are 514 bytes (v4+)
- **CircID**: Circuit Identifier
- **CMD**: Cell type - RELAY (3) or RELAY_EARLY (9)
- **Rx**: Recognised field (0x0000)
- **Digest**: seeded running hash (truncated SHA-1)



Notable Properties

- It allows **“Leaky Pipes”**, meaning that the **OP** can send a message to **any OR** in the circuit, **and vice versa**.

Notable Properties

- It allows **"Leaky Pipes"**, meaning that the **OP** can send a message to **any OR** in the circuit, **and vice versa**.

- An **OR either recognizes a cell** (via **Rx** & **Digest**), meaning it is the intended recipient, **or forwards the cell** further down the circuit.

Notable Properties

- It allows **"Leaky Pipes"**, meaning that the **OP** can send a message to **any OR** in the circuit, **and vice versa**.

- An **OR either recognizes a cell** (via **Rx** & **Digest**), meaning it is the intended recipient, **or forwards the cell** further down the circuit.

- The **malleability of AES-CTR** propagates through **multiple layers**, enabling a **de-anonymizing** attack called a **tagging attack**.

Notable Properties

Cryptography Research Centre

- It allows **"Leaky Pipes"**, meaning that the **OP** can send a message to **any OR** in the circuit, **and vice versa**.

- An **OR either recognizes a cell** (via **Rx** & **Digest**), meaning it is the intended recipient, **or forwards the cell** further down the circuit.

- The **malleability of AES-CTR** propagates through **multiple layers**, enabling a **de-anonymizing** attack called a **tagging attack**.

- **Bits** are flipped in a ciphertext at $\mathbf{OR_1}$ and reversed at $\mathbf{OR_3}$ to **test** whether **decryption succeeds** and ciphertexts belong to the **same circuit**.

Technology Innovation Institute

Historical Timeline of Evolving Solutions

[2012] Tor Proposal 202

- In **Proposal 202** Nick Matthewson, expressed the need to update the onion encryption in Tor.

[2012] Tor Proposal 202

- In **Proposal 202** Nick Matthewson, expressed the need to update the onion encryption in Tor.

- Specifically, it called for Onion Encryption that:

  1. Protects against **Tagging Attacks**,
  2. Is **non-expanding**, so as not to leak circuit size and relative position,
  3. Offers **Stronger end-to-end integrity** protection,
  4. Does not use **MAC-then-Encrypt**.

Proposed Solutions

- In **Prop 261(2016)** Matthewson proposed a new Onion Encryption scheme where each encryption layer consists of a **Wide-Input Tweakable Cipher.**

- Intuitively, **WITC** are **non-expanding**, **non-malleable**, and **integrity** can be added by appending **redundant bits** to the plaintext.

Proposed Solutions

- In **Prop 261(2016)** Matthewson proposed a new Onion Encryption scheme where each encryption layer consists of a **Wide-Input Tweakable Cipher.**

- Intuitively, **WITC** are **non-expanding**, **non-malleable**, and **integrity** can be added by appending **redundant bits** to the plaintext.

- **[ADL17]** Propose **GCM-RUP,** an AEAD scheme realized from GCM components that is **secure** under the **release of unverified plaintext**.

Proposed Solutions

- In **Prop 261(2016)** Matthewson proposed a new Onion Encryption scheme where each encryption layer consists of a **Wide-Input Tweakable Cipher.**

- Intuitively, **WITC** are **non-expanding**, **non-malleable**, and **integrity** can be added by appending **redundant bits** to the plaintext.

- **[ADL17]** Propose **GCM-RUP,** an AEAD scheme realized from GCM components that is **secure** under the **release of unverified plaintext**.

- In **[ADL17]** GCM-RUP is also proposed for **use in Tor**, claiming that a **WITC is overkill**, an OE scheme is described but **without any formal analysis**.

Onion Encryption from GCM-RUP

- In [**ADL17**] the Onion Encryption scheme was **underspecified**, **no integrity**, **no leaky pipes**, **no ciphertext chaining**, and **not forward secure**.

Onion Encryption from GCM-RUP

- In [**ADL17**] the Onion Encryption scheme was **underspecified**, **no integrity**, **no leaky pipes**, **no ciphertext chaining**, and **not forward secure**.

- [**Prop 295-Jul 2018**] Updated version **added leaky pipes** and **integrity**, **but not IND-CPA secure**, still no **ciphertext chaining** and **forward security**.

Onion Encryption from GCM-RUP

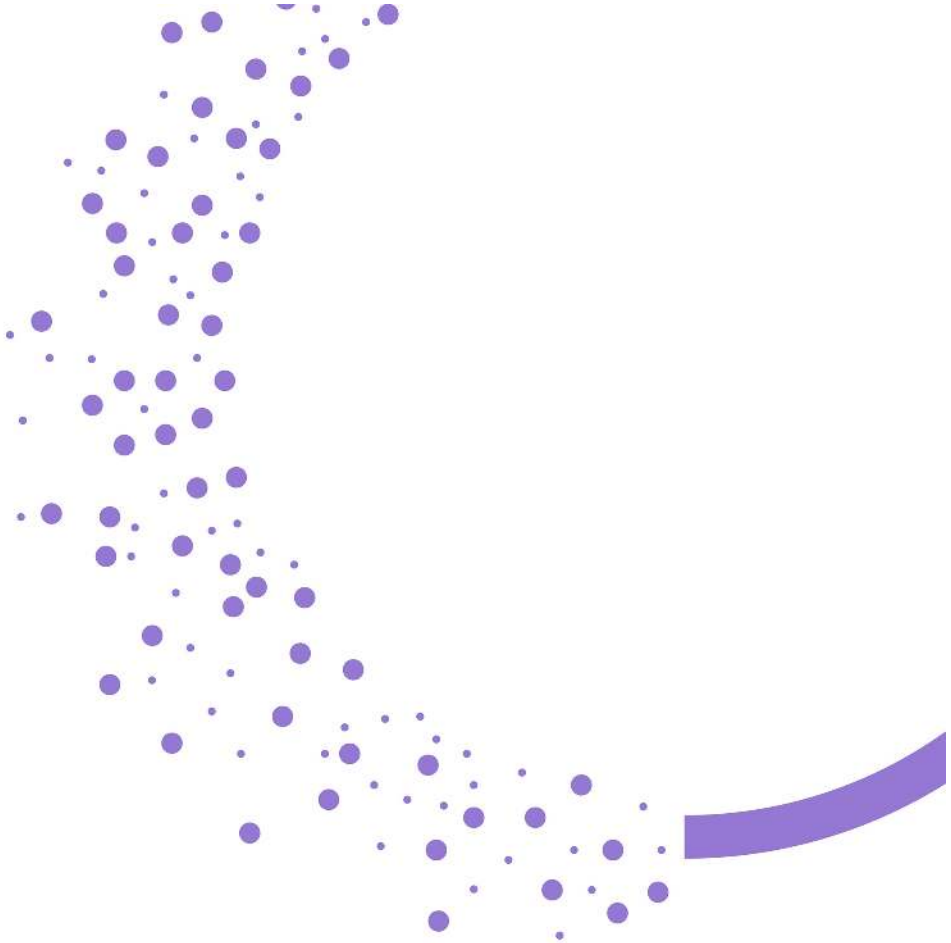
- In [**ADL17]** the Onion Encryption scheme was **underspecified**, **no integrity**, **no leaky pipes**, **no ciphertext chaining**, and **not forward secure**.

- [**Prop 295-Jul 2018]** Updated version **added leaky pipes** and **integrity**, **but not IND-CPA secure**, still no **ciphertext chaining** and **forward security**.

- **[Prop 295-Mar 2019]** Heavily revised scheme, made stateful by **feeding back GHASH outputs** (invalidating proofs) and **extending GCM-RUP to full WITC (SPRP),** forfeiting the performance gains originally sought!

Technology Innovation Institute

The Design of CGO



tii.ae

The Path Towards CGO

The Path Towards CGO

- We extracted a **length-preserving primitive** from GCM-RUP, which can be abstracted as a **Rugged PRP**.

The Path Towards CGO

- We extracted a **length-preserving primitive** from GCM-RUP, which can be abstracted as a **Rugged PRP**.

- Augmented the **Rugged PRP** construction with a **key update** functionality to achieve **forward security**.

The Path Towards CGO

- We extracted a **length-preserving primitive** from GCM-RUP, which can be abstracted as a **Rugged PRP**.

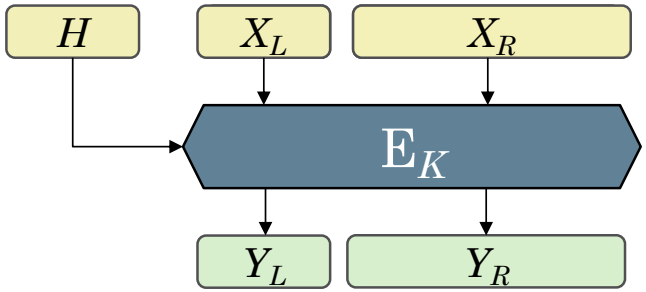
- Augmented the **Rugged PRP** construction with a **key update** functionality to achieve **forward security**.

- A new formal treatment of OE extending prior ones [DegSta18, RogZha18] with **Leaky Pipes**, **Associated Data**, **Forward Security**.

The Path Towards CGO

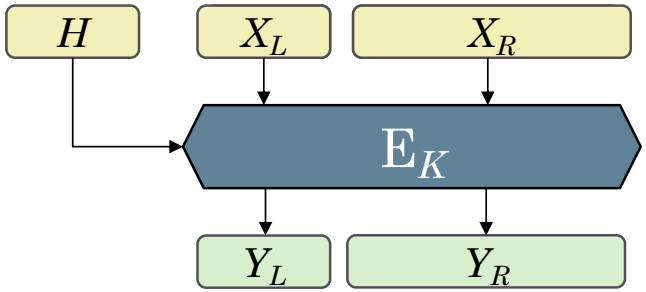
- We extracted a **length-preserving primitive** from GCM-RUP, which can be abstracted as a **Rugged PRP**.
- Augmented the **Rugged PRP** construction with a **key update** functionality to achieve **forward security**.
- A new formal treatment of OE extending prior ones [DegSta18, RogZha18] with **Leaky Pipes**, **Associated Data**, **Forward Security**.
- We build CGO in a modular fashion from any **Updateable Rugged PRP** and **prove it secure** in our new model.

Main Building Block: Rugged PRPs



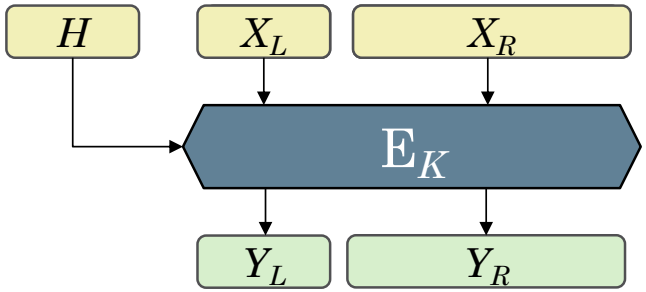
- A Rugged PRP is a **tweakable cipher** over a **split domain**: $\{0,1\}^n \times \{0,1\}^*, n = 128$.

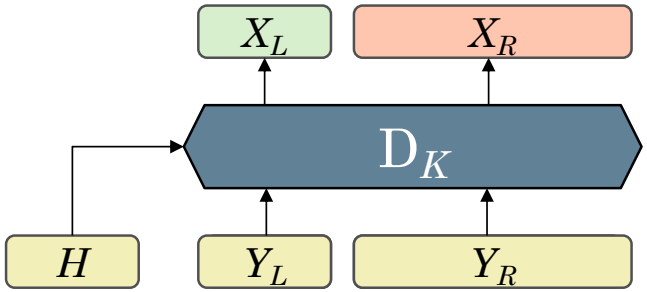
Main Building Block: Rugged PRPs



- A Rugged PRP is a **tweakable cipher** over a **split domain**: $\{0,1\}^n \times \{0,1\}^*, n = 128$.
- **Intermediate Security**, between PRP and SPRP security.
- **Very Informally**: Enciphering is **pseudorandom**, whereas Deciphering only provides **partial non-malleability**.

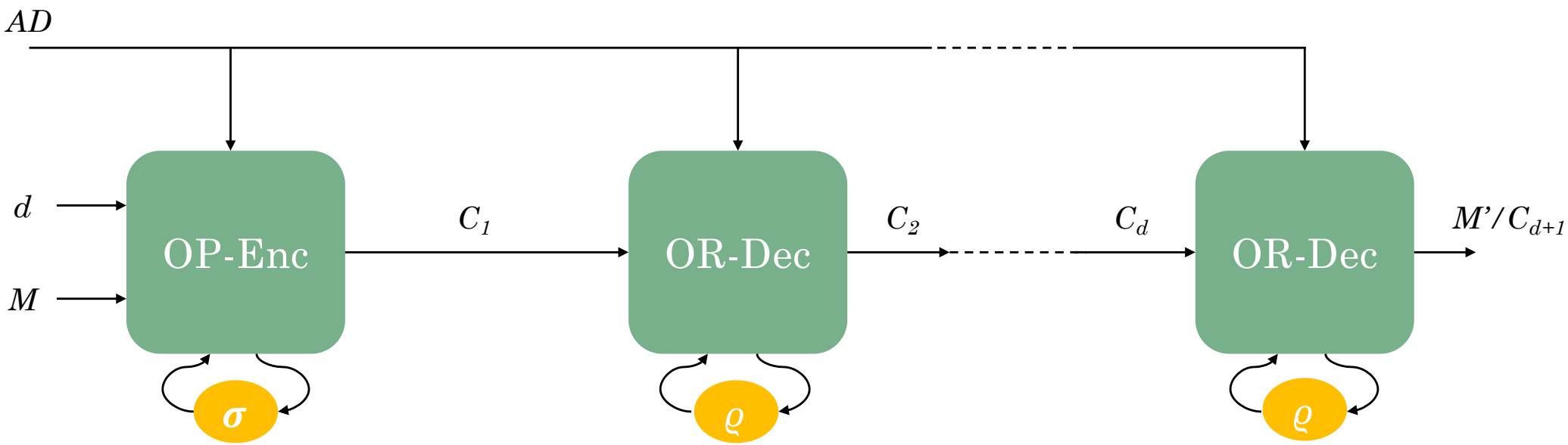
Main Building Block: Rugged PRPs



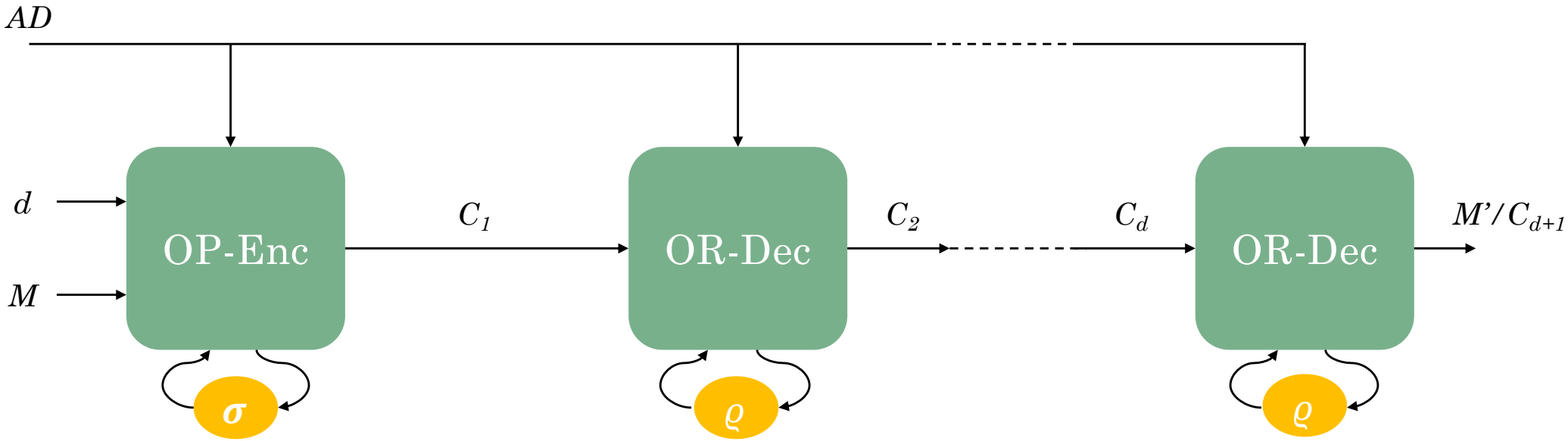


- A Rugged PRP is a **tweakable cipher** over a **split domain**: $\{0,1\}^n \times \{0,1\}^*, n = 128$.
- **Intermediate Security**, between PRP and SPRP security.
- **Very Informally**: Enciphering is **pseudorandom**, whereas Deciphering only provides **partial non-malleability**.

Syntax of Onion Encryption (Forward Direction)

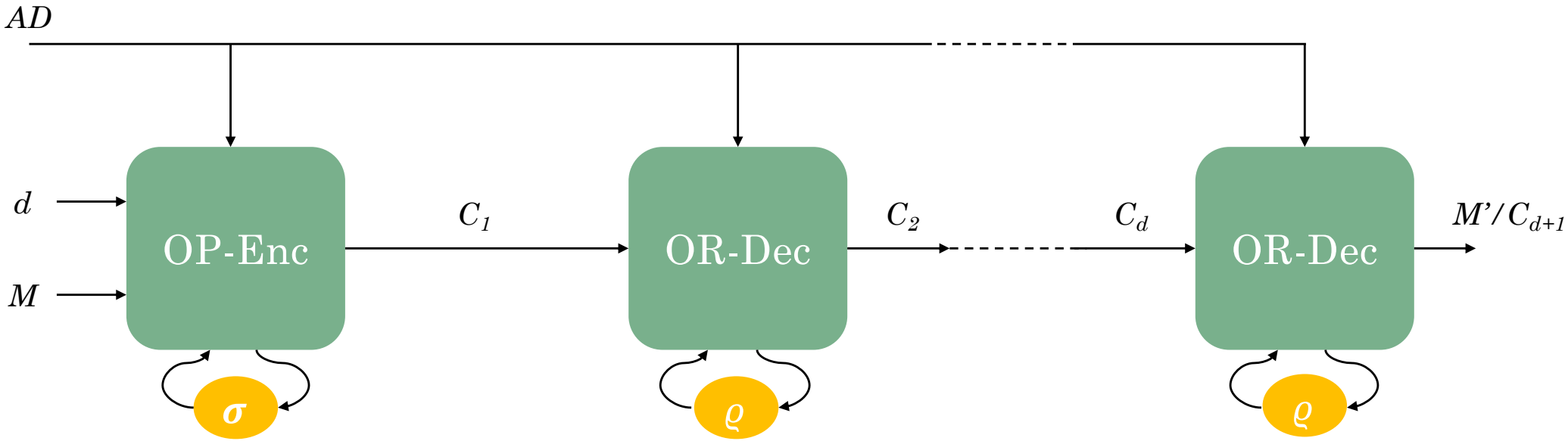


Syntax of Onion Encryption (Forward Direction)



- **Note:** decryption cannot detect invalid ciphertexts – **does not return** ⊥.

Syntax of Onion Encryption (Forward Direction)



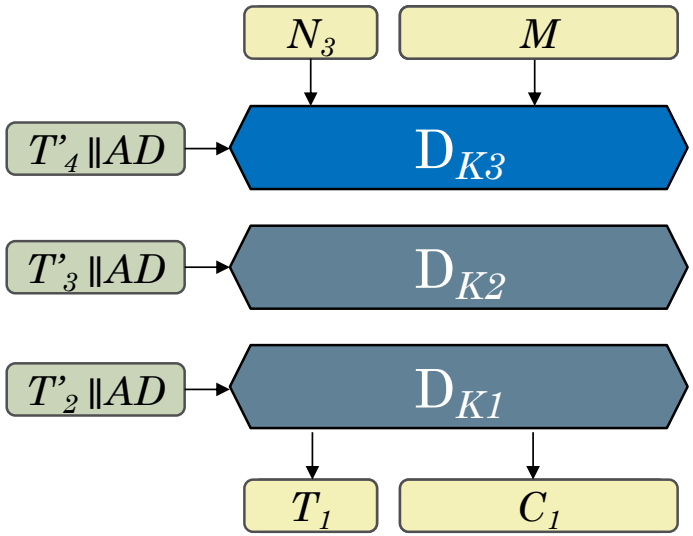
- **Note:** decryption cannot detect invalid ciphertexts – **does not return** ⊥.
- Hence the need to **chain ciphertexts** - if a **ciphertext is tampered** subsequent ciphertexts **decrypt to garbage**.

CGO (High-Level): Forward Direction

OP-Enc($\boldsymbol{\sigma}$,d=3,AD,M)$\rightarrow$ T_1,C_1 $\quad$ $\boldsymbol{\sigma}$ =**[**(K_i,N_i,T'_i)**]**

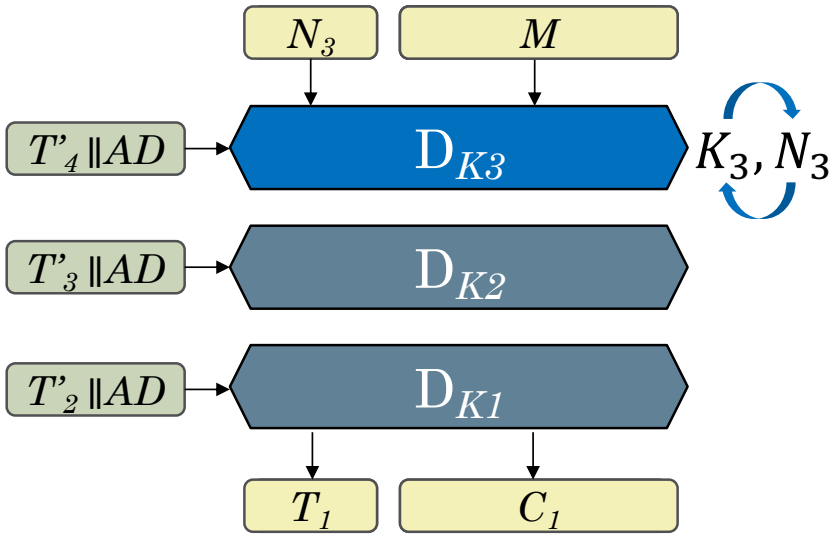
CGO (High-Level): Forward Direction

OP-Enc($\boldsymbol{\sigma}$,d=3,AD,M)$\rightarrow$ T_1,C_1 $\quad$ $\boldsymbol{\sigma}$ = **[**(K_i,N_i,T'_i)**]**



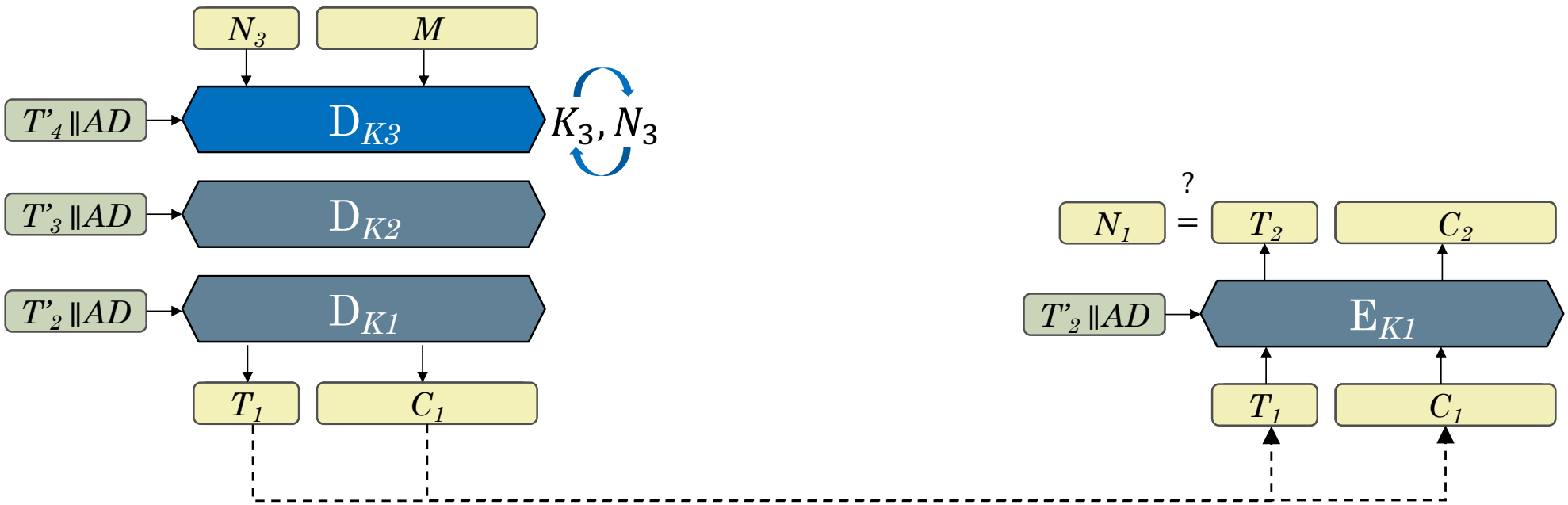
CGO (High-Level): Forward Direction

OP-Enc($\boldsymbol{\sigma}$,d=3,AD,M)$\rightarrow$ T_1,C_1 $\quad$ $\boldsymbol{\sigma}$ =[(K_i,N_i,T'_i)]



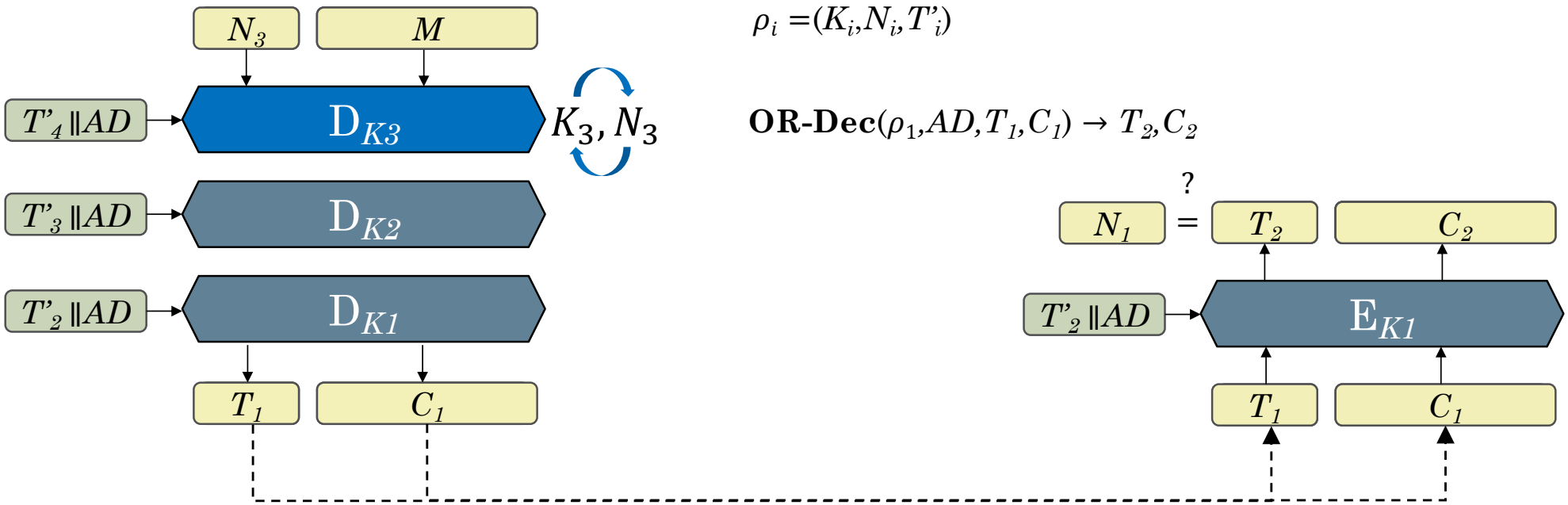
CGO (High-Level): Forward Direction

OP-Enc($\boldsymbol{\sigma}$,d=3,AD,M)→ T_1,C_1 $\boldsymbol{\sigma}$ =[(K_i,N_i,T'_i)]



CGO (High-Level): Forward Direction

OP-Enc($\boldsymbol{\sigma}$,d=3,AD,M)$\rightarrow T_1, C_1$ $\quad \boldsymbol{\sigma} = [(K_i, N_i, T'_i)]$

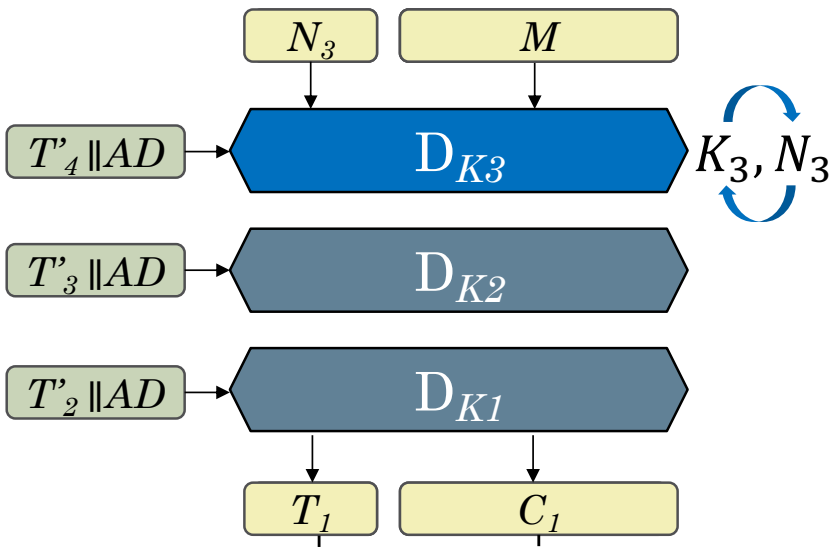


$\rho_i = (K_i, N_i, T'_i)$

OR-Dec(ρ_1,AD,T_1,C_1) $\rightarrow T_2, C_2$

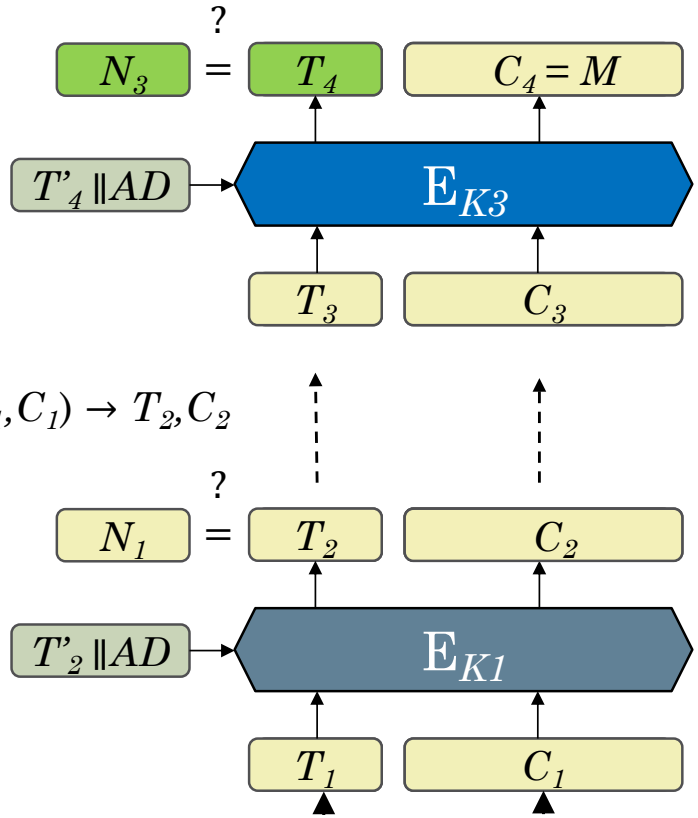
CGO (High-Level): Forward Direction

OP-Enc($\boldsymbol{\sigma}$,d=3,AD,M)→ T_1,C_1 $\boldsymbol{\sigma}$ =[(K_i,N_i,T'_i)]



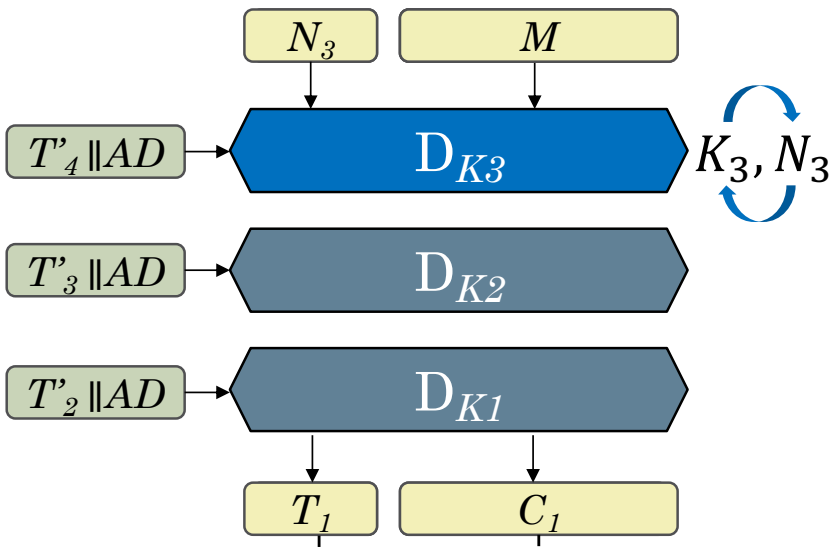
ρ_i =(K_i,N_i,T'_i)

OR-Dec(ρ_1,AD,T_1,C_1) → T_2,C_2



CGO (High-Level): Forward Direction

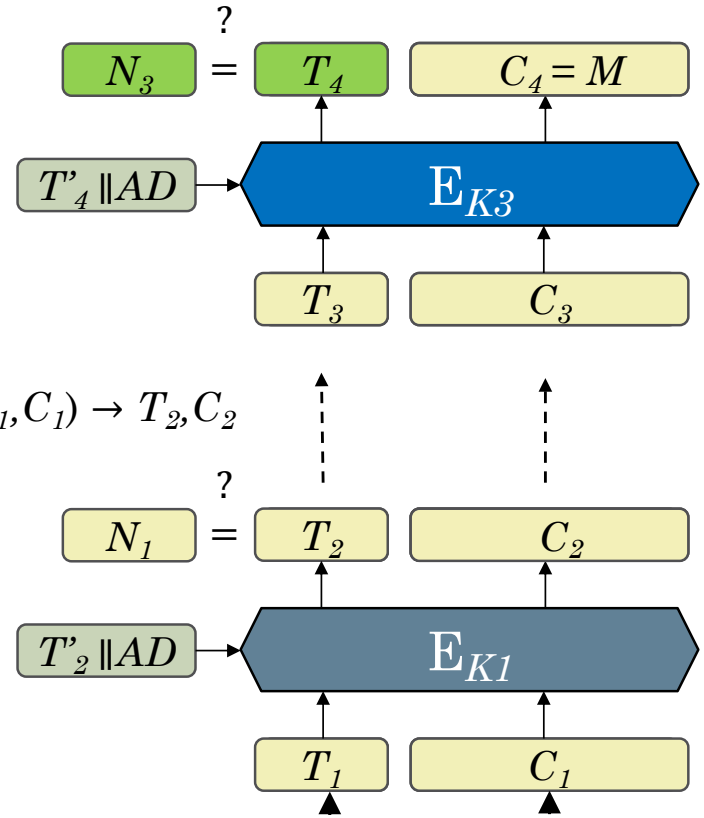
OP-Enc($\boldsymbol{\sigma}$,d=3,AD,M)$\rightarrow T_1, C_1$ $\quad \boldsymbol{\sigma} = [(K_i, N_i, T'_i)]$



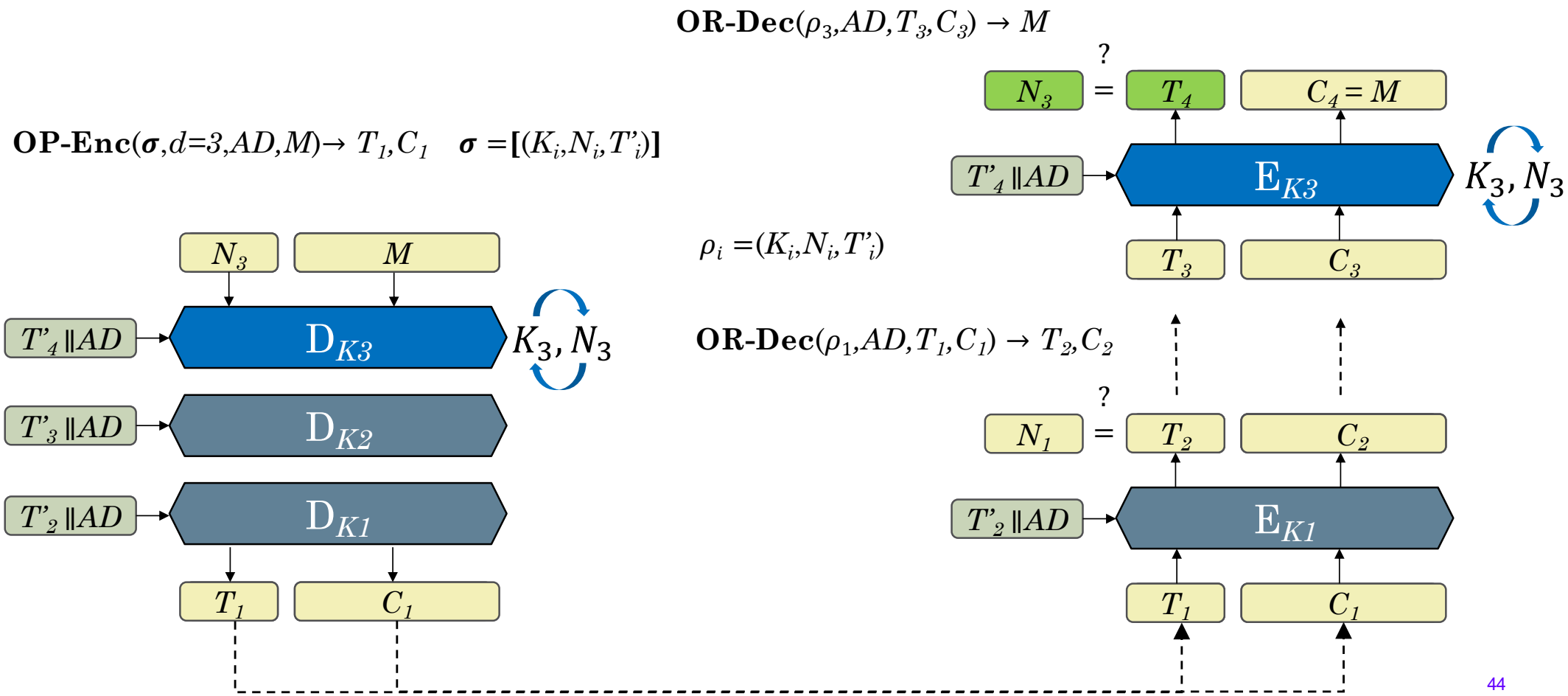
OR-Dec(ρ_3,AD,T_3,C_3) $\rightarrow M$

$\rho_i = (K_i, N_i, T'_i)$

OR-Dec(ρ_1,AD,T_1,C_1) $\rightarrow T_2, C_2$



CGO (High-Level): Forward Direction



High-Level Design Rationale

- **Non-malleability only where needed**—at the Onion Routers through RPRP enciphering, it is not needed at the Onion Proxy.

High-Level Design Rationale

- **Non-malleability only where needed**—at the Onion Routers through RPRP enciphering, it is not needed at the Onion Proxy.

- **The nonce** serves to : 1) **diversify** ciphertexts 2) non-repeating **left RPRP inputs** 3) provide **integrity** 4) support **leaky pipes**.

High-Level Design Rationale

- **Non-malleability only where needed**—at the Onion Routers through RPRP enciphering, it is not needed at the Onion Proxy.

- **The nonce** serves to : 1) **diversify** ciphertexts 2) non-repeating **left RPRP inputs** 3) provide **integrity** 4) support **leaky pipes**.

- The **tag** acts as a **compressed digest** of the ciphertext, allowing **efficient ciphertext chaining**–preventing recovery from invalid ciphertexts.

High-Level Design Rationale

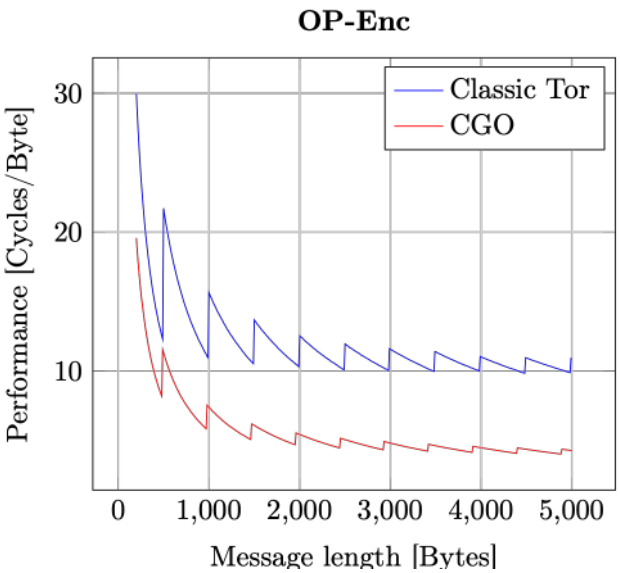
- **Non-malleability only where needed**—at the Onion Routers through RPRP enciphering, it is not needed at the Onion Proxy.

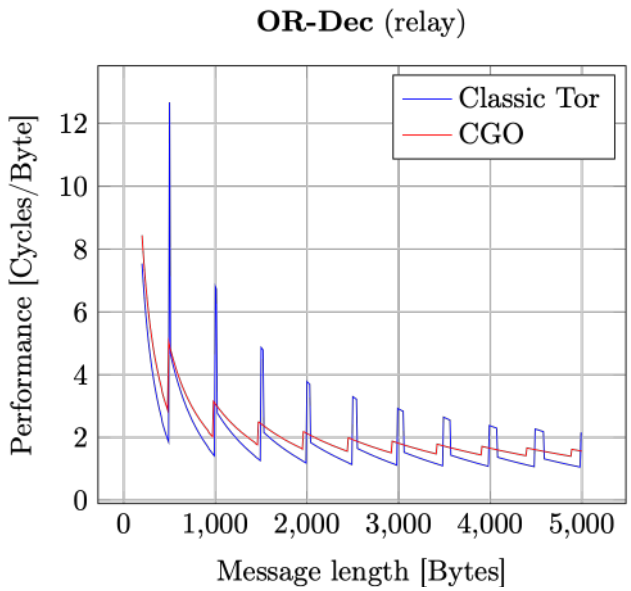
- **The nonce** serves to : 1) **diversify** ciphertexts 2) non-repeating **left RPRP inputs** 3) provide **integrity** 4) support **leaky pipes**.

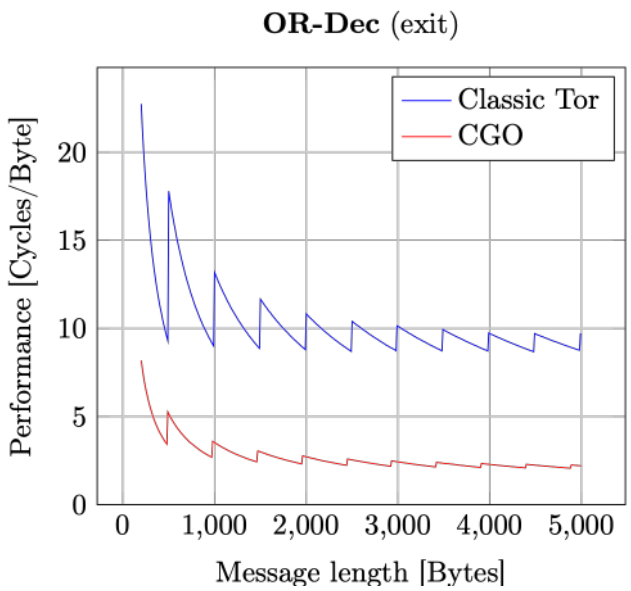
- The **tag** acts as a **compressed digest** of the ciphertext, allowing **efficient ciphertext chaining**–preventing recovery from invalid ciphertexts.

- **Associated Data**, prevents tagging in the headers (**CMU 2014 Attack**).

CGO Performance







Technology Innovation Institute

Summary

Summary

- Compared to Legacy Tor, CGO protects against **tagging attacks**, provides **stronger integrity** covering **associated data**, and adds **Forward Security**.
- All while maintaining a comparable (arguably better) **performance tradeoff**.

Summary

- Compared to Legacy Tor, CGO protects against **tagging attacks**, provides **stronger integrity** covering **associated data**, and adds **Forward Security.**
- All while maintaining a comparable (arguably better) **performance tradeoff**.
- CGO's **modular design** means that the **RPRP construction** can be easily replaced if needed, say with a **BBB construction**.

Summary

- Compared to Legacy Tor, CGO protects against **tagging attacks**, provides **stronger integrity** covering **associated data**, and adds **Forward Security.**
- All while maintaining a comparable (arguably better) **performance tradeoff**.
- CGO's **modular design** means that the **RPRP construction** can be easily replaced if needed, say with a **BBB construction**.
- CGO is currently being integrated in **Arti**—Tor's upcoming Rust implementation.
- https://eprint.iacr.org/2025/583
- https://eprint.iacr.org/2025/2017